



Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Universidad Nacional de Rosario

Tecnicatura Universitaria en Inteligencia Artificial

Procesamiento del Lenguaje Natural

Trabajo Práctico N.º 2

Embeddings y búsqueda semántica

Grupo 7

Civetta, Valentino	Legajo 45948755
Frank, Maximiliano	Legajo 38726401
Fucci, Milagros	Legajo 47076861
Winter, Federico	Legajo 44101265

Docentes: Geary, Alan y Manson, Juan Pablo

Octubre de 2026

1. Qué hicimos

Partimos del corpus del TP1: 150 sinopsis de la categoría “Clásico” de Lectulandia. Sobre ese corpus probamos tres formas de convertir texto en vectores y las comparamos con TF-IDF, que fue lo que usamos en el TP1.

Cada modelo recibe el texto que le corresponde. SBERT lee la sinopsis cruda, con mayúsculas y puntuación, porque aprendió sobre oraciones reales. Word2Vec y TF-IDF leen una versión limpia, en minúsculas y sin puntuación ni stopwords, porque solo miran qué palabras aparecen.

Los dos modelos de palabras son un Word2Vec que entrenamos nosotros (Skip-gram, 100 dimensiones, semilla fija) y SBW, un modelo ya entrenado sobre mucho texto en español (300 dimensiones). En los dos casos el vector de un libro es el promedio de los vectores de sus palabras. Es la forma más simple de pasar de palabras a documentos, aunque pierde el orden, las negaciones y el peso de cada palabra. El tercer modelo es SBERT (distiluse-base-multilingual-cased-v1, 512 dimensiones), que arma un vector para la oración entera. Lee como máximo 128 tokens, y 134 de las 150 sinopsis son más largas, así que se cortan.

Guardamos los vectores del Word2Vec propio y de SBERT en Supabase con pgvector, en una tabla por modelo porque una columna vector(n) tiene la dimensión fija. Los normalizamos antes de insertarlos, no quedó ninguno nulo y creamos índices HNSW con vector_cosine_ops, que es la opción que corresponde al operador <=> que usamos en las consultas. La búsqueda se resuelve en SQL y se puede filtrar por género.

Para controlar la base comparamos el top-5 de SQL con el que calculamos en numpy. Coincidieron en todas las consultas salvo una, por un empate entre las dos ediciones de *Las mil y una noches*: sus sinopsis son iguales en los primeros 128 tokens, así que SBERT les da el mismo vector, y cada sistema desempata a su manera. La misma búsqueda sin índice dio el mismo resultado, así que el índice HNSW no está perdiendo libros.

2. Evaluación

Escribimos 12 consultas (queries.json) leyendo solo las sinopsis y antes de correr ningún modelo. Hay 3 léxicas, que comparten palabras con los libros buscados; 5 semánticas, que los describen con otras palabras; y 4 que no tienen ninguna palabra en común con sus relevantes, cosa que verificamos comparando palabras y raíces. Cada consulta tiene entre 2 y 7 libros relevantes, contando todas las ediciones de un mismo libro.

Método	P@5	P@10	R-prec.	R-prec. léxicas	R-prec. semánticas	R-prec. sin palabras comunes
Azar (esperado)	0,025	0,025	0,025	0,029	0,025	0,022
TF-IDF	0,283	0,217	0,357	0,746	0,410	0,000
Word2Vec propio	0,167	0,125	0,159	0,270	0,129	0,112
SBW (promedio)	0,267	0,208	0,286	0,571	0,202	0,175
SBERT	0,350	0,258	0,466	0,857	0,414	0,238

Para saber cuánto puede sacar el azar, además del valor esperado (0,025) simulamos 10.000 buscadores que devuelven libros al azar. El percentil 95 de su P@5 promedio es 0,067, y todos los métodos quedan por encima.

3. Respuestas a las preguntas de la consigna

¿Qué modelo usaríamos en producción? SBERT. Tiene el mejor promedio y es el único que no queda por debajo de TF-IDF en ningún tipo de consulta. Además corre en una computadora común sin GPU, no tiene costo por consulta, las consultas no salen de nuestro servidor y, con los pesos fijos, siempre da el mismo resultado. De terceros solo depende para la descarga inicial desde Hugging Face. Su punto débil es el corte a 128 tokens; lo resolveríamos partiendo las sinopsis en fragmentos y mantendríamos TF-IDF como apoyo para las consultas con nombres propios. Descartamos el

Word2Vec propio porque 150 documentos no alcanzan para entrenarlo, y SBW, que sería la segunda opción, necesita 1,1 GB de vectores para dar un resultado peor. Supabase es un servicio externo, pero pgvector se puede instalar en cualquier Postgres propio.

¿Cuánto mejor que TF-IDF? En promedio, SBERT sube la R-precision de 0,357 a 0,466. Si comparamos consulta por consulta, gana en 5, empata en 5 y pierde en 2. Con un test de signos eso da $p \approx 0,45$, así que con 12 consultas no podemos decir que la diferencia sea real. Lo que sí se ve es dónde se separan. En las consultas sin palabras en común TF-IDF saca 0 (no puede encontrar nada si no hay palabras compartidas) y SBERT llega a 0,24. En las semánticas empatan, con 0,41 cada uno. En las léxicas esperábamos que ganara TF-IDF y ganó SBERT, 0,86 contra 0,75. En la consulta sobre las guerras napoleónicas ningún método puso un relevante en el top-5.

¿Qué mide la métrica y qué no? precision@k cuenta cuántos de los libros que nosotros marcamos como relevantes aparecen entre los primeros k. No tiene en cuenta el orden dentro de esos k, no da puntos por un libro que es relevante a medias (*Germinal* para “lucha de clases” cuenta como error) y no dice si a un usuario real le habría servido. Además, una consulta con 2 relevantes nunca puede pasar de 0,4 en P@5; por eso reportamos también R-precision. Las consultas y los juicios de relevancia los escribimos nosotros, y son solo 12, así que las conclusiones son tentativas.

Un caso de falla. La consulta “obra de teatro sobre un gobernante que pierde la razón o acaba atormentado por sus actos” tiene como relevantes las dos ediciones de *Macbeth* y *El rey Lear*. SBERT devolvió *Preparación del actor*, *Julio César*, *La vida es sueño*, *Hamlet* y recién quinta una edición de *Macbeth*. Entendió “teatro” y “rey”, pero no la trama. Nuestra hipótesis fue que el modelo nunca leyó la parte de la sinopsis que hace relevantes a esos libros, y la verificamos con su tokenizador. En la otra edición de *Macbeth*, “tormento” y “remordimientos” empiezan en los tokens 367 y 383, y en *El rey Lear* “perder la razón” empieza en el 292: todo eso queda fuera de los 128 que lee SBERT. La edición de *Macbeth* que sí encontró habla del “mal que nace del ansia de poder” en el token 45. Parte del error también es nuestro, porque *Julio César* y *Hamlet* eran candidatos dudosos que dejamos afuera con un criterio estricto.

4. Parte avanzada: RAG

Para cada pregunta recuperamos con SBERT los 4 libros más parecidos y le pasamos sus sinopsis completas a un modelo generativo de la API de NVIDIA NIM. Probamos dos prompts. El v1 solo pide responder usando los libros. El v2 pone reglas: usar solo el contexto, citar el documento de cada dato como [Dn], abstenerse con una frase fija cuando no hay información y avisar cuando la pregunta parte de algo falso. Escribimos 9 preguntas antes de correrlo (4 con respuesta en el catálogo, 2 sin respuesta, 2 con una premisa falsa y 1 sobre un error del corpus) y revisamos a mano las 18 respuestas del modelo google/gemma-4-31b-it.

No encontramos alucinaciones con ninguno de los dos prompts. Cuando el libro con la respuesta fue recuperado, los dos respondieron bien, y los dos corrigieron las premisas falsas. La diferencia del v2 fue de formato: siempre se abstuvo con la misma frase y citó cada dato, lo que permite controlarlo automáticamente. El v1 respondió bien, pero con frases variables que nuestro chequeo automático no reconocía.

Todas las fallas vinieron de la recuperación. *Morfina* quedó en el puesto 12 porque la palabra “morfina” aparece en el token 289 de su sinopsis, otra vez por el corte de SBERT. Las preguntas que nombran un autor o un título (Camus, *Middlemarch*) no traen esos libros porque solo indexamos la sinopsis, así que la pregunta sobre el error del corpus nunca llegó al modelo con el libro correcto.

También vimos que ser fiel al contexto no garantiza decir la verdad. El v1 afirmó que “ninguno de los libros del catálogo” trataba sobre la morfina cuando solo había visto 4, y nuestra frase de abstención (“no hay información suficiente en el catálogo”) tiene el mismo problema. Por último, mientras hacíamos el TP NVIDIA retiró de su API todos los modelos Llama de Meta. El código ahora elige el modelo de una lista de preferencia y deja registrado cuál usó, pero mientras el modelo no sea nuestro el resultado no es del todo reproducible. Como mejoras, partiríamos las sinopsis en fragmentos, indexaríamos también título y autores, y cambiaríamos la abstención para que hable de “los documentos recuperados”.

5. Otros hallazgos

El Word2Vec propio no aprendió casi nada. La similitud media entre dos libros al azar es 0,999, el 96 % de la varianza entra en 2 componentes de la PCA y sus vecinos no tienen sentido (para “amor” devuelve “vida”, “así” y “obra”). Además no conoce la mayoría de las palabras de las consultas difíciles. En una corrida anterior, sin semilla fija, su P@5 fue 0,067, igual al percentil 95 del azar; con la semilla 42 da 0,167. Con tan pocos datos, su resultado depende más del azar del entrenamiento que del corpus.

En el corpus encontramos dos problemas. La sinopsis de *Middlemarch* es en realidad un fragmento del prólogo del *Lazarillo de Tormes*, y siete libros aparecen en dos ediciones. Ninguna métrica de búsqueda detecta el primero.

6. Uso de asistentes de IA

Usamos Claude (Anthropic) como asistente durante el trabajo: para configurar Supabase y el usuario de conexión, escribir y depurar el código de evaluación y del RAG, proponer borradores de las consultas a partir de las sinopsis y redactar una primera versión de este informe y de los textos del notebook. Las consultas, sus relevantes y las preguntas del RAG los revisamos y aprobamos nosotros antes de correr los modelos, y la interpretación de los resultados la discutimos en el grupo. Somos responsables de todo lo que entregamos.